

PAM, un modulo alla volta

Lo stack che decide se entri. Leggerlo, chiamarlo direttamente, e poi cambiarlo sette volte — regole sulle password, blocco dell'account, limiti, restrizioni per host, un hook di audit e un server di directory. Da un lab in esecuzione.

Comando `qlab run pam-lab`

VM `pam-lab-server` · `pam-lab-client` · `pam-lab-ldap`

LAN `192.168.100.0/24`, isolata fra le tre VM

Credenziali `labuser / labpass` · `testuser / Test123!` · `alice / Alice123!`

Risultato `qlab test pam-lab` → 8 esercizi, 43 controlli, tutti superati

1. Tre macchine, un login

```
pam-lab-server      192.168.100.1/24
pam-lab-client      192.168.100.2/24
pam-lab-ldap        192.168.100.3/24
```

```
testuser:1001:/bin/bash
alice:1002:/bin/bash
```

Il server è dove si configura PAM. Il client esiste perché i login arrivino dalla rete, da un indirizzo che il server può vedere e su cui può decidere: parecchi dei moduli qui sotto hanno senso solo se il login viene da qualche parte. Il server LDAP serve per l'ultima sezione, e fino ad allora si ignora.

Tutto quello che segue cambia la configurazione PAM del server e poi la rimette a posto. Vale la pena dirlo ad alta voce: PAM è l'unico sottosistema in cui un refuso ti chiude fuori dalla tua stessa macchina. È il motivo per cui ogni esperimento qui è accoppiato a un ripristino, ed è il motivo per cui `pamtester` — sezione 3 — esiste.

2. Lo stack è un file di testo

```
$ cat /etc/pam.d/sshd      (comments and blank lines removed)
@include common-auth
account    required      pam_nologin.so
@include common-account
session [success=ok ignore=ignore module_unknown=ignore default=bad]      pam_selinux.so close
session    required      pam_loginuid.so
session    optional      pam_keyinit.so force revoke
@include common-session
session    optional      pam_mail.so standard noenv # [1]
session    required      pam_limits.so
session    required      pam_env.so # [1]
session    required      pam_env.so user_readenv=1 envfile=/etc/default/locale
session [success=ok ignore=ignore module_unknown=ignore default=bad]      pam_selinux.so open
@include common-password

$ the file it @includes for authentication
auth       [success=2 default=ignore] pam_unix.so nullok
auth       [success=1 default=ignore] pam_sss.so use_first_pass
auth       requisite      pam_deny.so
auth       required       pam_permit.so
auth       optional       pam_cap.so

$ and how many services on this machine have a PAM stack of their own
chfn chpasswd chsh common-account common-auth common-password common-session
common-session-noninteractive login newusers other passwd polkit-1 runuser
runuser-l sshd sssd-shadowutils su su-l sudo sudo-i
```

`/etc/pam.d/sshd` non è un file di configurazione *sull'*autenticazione: è l'autenticazione, in ordine. Ogni riga ha tre parti: un **tipo**, un **controllo** e un **modulo**.

I quattro tipi sono quattro stack separati che girano in momenti diversi. `auth` dimostra chi sei. `account` decide se quell'identità può entrare *adesso, qui* — valida, non scaduta, non bloccata.

`session` prepara e smonta quello che sta intorno al login. `password` gira solo quando si cambia una credenziale. Un modulo può implementarne un sottoinsieme qualsiasi, ed è il motivo per cui lo stesso `pam_unix.so` compare in più stack facendo lavori diversi.

Il campo di controllo è quello interessante. `required`, `requisite`, `sufficient` e `optional` sono le forme abbreviate; `common-auth` mostra la forma lunga in cui si espandono: `[success=2 default=ignore]` significa *se riesce, salta le due righe successive*. È così che si esprime "prova gli utenti locali, poi prova la directory, poi arrenditi" senza un solo `if`.

Le righe `@include` contano quanto il resto: il file del singolo servizio delega ai `common-*`, ed è per questo che installare qualcosa che modifica `common-auth` cambia il modo in cui si autentica *ogni* servizio della macchina.

3. Chiamare PAM direttamente

```
$ pamtester sshd testuser authenticate      - right password
pamtester: invoking pam_start(sshd, testuser, ...)
pamtester: performing operation - authenticate
Password: pamtester: successfully authenticated

$ the same call, wrong password
pamtester: invoking pam_start(sshd, testuser, ...)
pamtester: performing operation - authenticate
Password: pamtester: Authentication failure

$ and the four stacks of the same service, asked one at a time
authenticate Password: pamtester: successfully authenticated
acct_mgmt     pamtester: account management done.
open_session  pamtester: successfully opened a session
close_session pamtester: session has successfully been closed.
```

`pamtester` esegue lo stack come farebbe un servizio, senza esserlo. La prima chiamata autentica `testuser` contro lo stack `sshd` — nessun demone SSH di mezzo, nessuna rete, solo la libreria. La seconda fa lo stesso con una password sbagliata e ottiene il fallimento.

Le ultime quattro chiamate sono i quattro tipi della sezione 2, richiesti uno per volta sullo stesso servizio. `authenticate` controlla la password, `acct_mgmt` chiede se l'account è utilizzabile, `open_session` e `close_session` eseguono gli hook di sessione. Ogni login che hai fatto in vita tua è queste quattro chiamate in quest'ordine, fatte da un programma che poi va per la sua strada.

Questo è lo strumento che rende PAM sperimentabile in sicurezza. Apri un secondo terminale già collegato, cambia lo stack, provalo con `pamtester`, e solo dopo tenta un login vero. Modificare `common-auth` e uscire per vedere se ha funzionato è il modo in cui si finisce a cercare la console di ripristino.

4. Regole sulle password valide per qualcuno

```
$ the rules being added to /etc/security/pwquality.conf
# == walkthrough ==
minlen = 12
minclass = 3
maxrepeat = 3
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1

$ testuser changing his own password – three tries in one session
Changing password for testuser.
Current password: New password: BAD PASSWORD: The password contains less than 1 digits
New password: BAD PASSWORD: The password contains less than 1 digits
New password: Retype new password: passwd: password updated successfully

$ and root setting the very same weak password on that account
New password: BAD PASSWORD: The password contains less than 1 digits
Retype new password: passwd: password updated successfully
(rules removed, password restored)
```

`pam_pwquality` sta in cima allo stack `password` e passa al setaccio la nuova password prima che `pam_unix` sia autorizzato a scriverla. Le regole stanno in `/etc/security/pwquality.conf`: una lunghezza minima, un numero minimo di classi di caratteri e le impostazioni `*credit`, dove un valore **negativo** significa "richiedine almeno tanti" — `dcredit = -1` è una cifra, minimo.

Quando `testuser` cambia la propria password le regole sono applicate: due rifiuti con la motivazione scritta per esteso, poi una password che soddisfa tutto, accettata. `retry = 3` è il motivo per cui ha tre tentativi prima che `passwd` si arrenda del tutto.

Poi root imposta la stessa password rifiutata sullo stesso account, e passa — con la lamentela stampata, e ignorata. `pam_pwquality` non applica le regole a root se non gli si dà `enforce_for_root`. Questa sorprende regolarmente: una policy provata con `sudo passwd unutente` sembra non funzionare affatto, perché per root è sempre e solo un consiglio.

5. Blocco: tre colpi, contati su disco

```
$ the right password, before anything is changed
login succeeded

$ the auth stack, rebuilt around pam_faillock
auth    required      pam_faillock.so preauth silent deny=3 unlock_time=300
auth    [success=3 default=ignore] pam_unix.so nullok
auth    [default=die] pam_faillock.so authfail deny=3 unlock_time=300
auth    requisite     pam_deny.so
auth    required      pam_permit.so

account required pam_faillock.so

$ three wrong passwords from the client...
attempt 1: refused
attempt 2: refused
attempt 3: refused
$ ...and then the right one
refused
```

```

$ what the server made of those four attempts
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser
pam_faillock(sshd:auth): Consecutive login failures for user testuser account temporarily locked

$ faillock --user testuser      (on the server)
testuser:
When                Type  Source                Valid
2026-09-14 14:20:49 RHOST 192.168.100.2        V
2026-09-14 14:20:52 RHOST 192.168.100.2        V
2026-09-14 14:20:55 RHOST 192.168.100.2        V

$ faillock --user testuser --reset, then the right password again
login succeeded
(stack restored)

```

Lo stack ricostruito mostra la forma di cui `pam_faillock` ha bisogno, e l'ordine non è facoltativo. `preauth` gira *prima* di `pam_unix` e rifiuta subito se l'account è già bloccato. `authfail` gira *dopo* e registra il fallimento — con `[default=die]`, così un tentativo fallito ferma lì lo stack. Metti `authfail` per primo e ogni login fallisce: è un errore genuinamente facile da fare.

Poi tre password sbagliate dal client, e il quarto tentativo — con la password *giusta* — viene rifiutato anche lui. L'account è bloccato, e il log del server lo dice con tutte le lettere: tre fallimenti di autenticazione di `pam_unix`, poi `pam_faillock` che annuncia che l'account è temporaneamente bloccato.

`faillock --user` stampa il registro: una riga per fallimento, con l'ora e l'indirizzo di provenienza, perché il contatore è per utente e vive in `/var/run/faillock/`. `--reset` lo svuota, e la stessa password rifiutata un attimo prima funziona di nuovo. `unlock_time=300` avrebbe fatto lo stesso da solo dopo cinque minuti.

6. I limiti arrivano con la sessione

```

$ limits.conf before and after, and what a login session gets
nofile for this session: 1024
pam_limits lines in common-session: 1

$ adding a per-user limit
testuser hard nofile 256
testuser soft nofile 128
testuser hard nproc 64

$ a fresh login session as testuser, from the client
128
256
64
(soft nofile, hard nofile, max user processes)
(limits.conf restored)

```

`pam_limits` è già in `common-session` — c'è di default in quasi tutte le distribuzioni — e applica `/etc/security/limits.conf` nel momento in cui una sessione si apre. Nient'altro nel sistema fa questa cosa: un limite messo qui è una proprietà del *fare login*, non della shell né dell'account.

Tre righe producono tre risposte diverse nella nuova sessione: un limite soft, uno hard e un tetto ai processi. Quello soft è ciò che un programma ottiene; quello hard è il soffitto fino al quale può alzarsi da solo, e non oltre. La distinzione è ciò che fa sì che `ulimit -n 4096` a volte funzioni e a volte no, sulla stessa macchina, per utenti diversi.

La misura deve venire da un login *nuovo*. Una shell già aperta si tiene i limiti con cui è nata, e nessuna modifica a `limits.conf` la cambierà.

7. Da dove stai entrando

```
$ a deny rule in /etc/security/access.conf, and the module that reads it
-:testuser:192.168.100.2
account required pam_access.so

$ testuser logging in from 192.168.100.2 – the client
Connection closed by 192.168.100.1 port 22

$ what the server logged about it
pam_access(sshd:account): access denied for user `testuser' from `192.168.100.2'

$ the same account, logging in on the server itself
pamtester: account management done.
(access.conf and the stack restored)
```

Una riga di `/etc/security/access.conf` — nega, questo utente, da quell'indirizzo — e `pam_access` nello stack `account` a leggerla. Il login del client viene chiuso prima di ottenere una shell, con la password perfettamente corretta, e il log registra esattamente perché.

È la dimostrazione più limpida del lab di *a cosa serve* lo stack `account`. L'autenticazione è riuscita: la password era giusta e `pam_unix` l'ha detto. L'autorizzazione è poi fallita, separatamente, per motivi che non hanno niente a che vedere con le credenziali. Due domande diverse, due stack diversi, e PAM li tiene distinti apposta.

Lo stesso account passa ancora `acct_mgmt` quando il login non arriva dall'indirizzo negato: la regola nomina una sorgente, quindi vale solo lì.

8. Tutto quello che puoi scrivere in uno script

```
$ a shell script bolted onto the session stack with pam_exec
session optional pam_exec.so /usr/local/bin/pam-audit.sh
#!/bin/bash
echo "$(date +%F %T)" PAM_TYPE=$PAM_TYPE USER=$PAM_USER RHOST=$PAM_RHOST SERVICE=$PAM_SERVICE
TTY=$PAM_TTY" >> /var/log/pam-audit.log

$ one SSH login from the client...
hello

$ ...and what the script wrote
```

```
2026-09-14 14:21:40 PAM_TYPE=open_session USER=testuser RHOST=192.168.100.2 SERVICE=sshd TTY=ssh
2026-09-14 14:21:40 PAM_TYPE=close_session USER=testuser RHOST=192.168.100.2 SERVICE=sshd TTY=ssh
2026-09-14 14:21:40 PAM_TYPE=open_session USER=labuser RHOST=10.0.2.2 SERVICE=sshd TTY=ssh
(sshd stack restored)
```

`pam_exec` esegue un programma qualsiasi come parte dello stack, passandogli il contesto in variabili d'ambiente. Dodici righe di shell e una riga in `/etc/pam.d/sshd` producono una traccia di audit dei login: chi, da dove, quale servizio, quale TTY, e se la sessione si stava aprendo o chiudendo.

Dopo un login il log ha tre voci, non due, e la terza è la connessione SSH della guida stessa che arriva a leggere il file. Vale la pena notarlo: l'hook è sul servizio, non sull'utente, e vede tutto quello che il servizio fa — compresa la cosa che stai facendo in questo momento.

`optional` è il controllo giusto qui: un hook di audit che può *far fallire il login* perché uno script aveva un refuso è un pessimo affare. Con `required`, un'uscita diversa da zero di quello script chiuderebbe fuori tutti.

9. Utenti che abitano altrove

```
$ before: does this machine know who ldapuser1 is?
passwd:      files systemd sss
group:       files systemd sss
shadow:      files sss
inactive
ldapuser1:*:10001:10000:LDAP User1:/home/ldapuser1:/bin/bash

$ the directory itself, on 192.168.100.3
dn: uid=ldapuser1,ou=users,dc=pam-lab,dc=local
uid: ldapuser1
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/ldapuser1
loginShell: /bin/bash

$ pointing the server at it: /etc/sss/sss.conf
passwd:      files sss
group:       files sss
shadow:      files sss
active

$ after: the same questions, same machine
ldapuser1:*:10001:10000:LDAP User1:/home/ldapuser1:/bin/bash
uid=10001(ldapuser1) gid=10000(pamtesters) groups=10000(pamtesters)
pamtesters:*:10000:ldapuser1,ldapuser2

lines mentioning ldapuser1 in /etc/passwd: 0

$ and ldapuser1 logging in over SSH, from the client
ldapuser1
/home/ldapuser1
. . . .bash_logout .bashrc .profile
(sssd stopped, nsswitch and the session stack restored)
```

`nsswitch.conf` elenca già `sss` come sorgente per `passwd`, `group` e `shadow` — ma `sssd` non è in esecuzione e non ha configurazione, quindi la macchina non ha mai avuto nessuno a cui chiedere. `getent passwd ldapuser1` torna a mani vuote.

La voce esiste, a una VM di distanza, in una directory LDAP: un `uid`, un `uidNumber`, un `gidNumber`, una home e una shell di login — gli stessi campi che ha `/etc/passwd`, scritti come oggetto di directory.

Un file di configurazione e un riavvio dopo, la macchina risponde diversamente alle stesse domande. `getent` risolve l'utente, `id` risolve l'appartenenza al gruppo, e `/etc/passwd` non contiene traccia né dell'uno né dell'altro. Poi `ldapuser1` entra via SSH dal client, e `pam_mkhomedir` gli crea la home al volo, copiandoci dentro `/etc/skel`.

È tutto qui il senso dell'esercizio: l'**identità** è arrivata dalla directory via NSS, l'**autenticazione** è arrivata dalla directory via `pam_sss`, e la **home** è arrivata da un modulo di sessione PAM — tre meccanismi separati che insieme trasformano un utente mai creato su questa macchina in un utente ordinario.

10. Provaci

```
qlab run pam-lab
qlab shell pam-lab-server
```

Il giro sicuro è sempre lo stesso:

```
sudo cp /etc/pam.d/common-auth /root/common-auth.bak # prima, sempre
sudo nano /etc/pam.d/common-auth
echo 'Test123!' | sudo pamtester -v sshd testuser authenticate
```

e dal client, un login vero per conferma:

```
sshpass -p 'Test123!' ssh testuser@192.168.100.1
```

Tre cose che vale la pena fare:

- Cambiare un `required` in `requisite` dentro `common-auth` e guardare dove si ferma lo stack: `requisite` fallisce subito, `required` finisce comunque lo stack così che un attaccante non possa capire *quale* modulo ha rifiutato.
- Impostare `deny=3` in `pam_faillock` e chiudersi fuori apposta, poi sbloccare l'account da una seconda sessione con `faillock --reset`.
- Aggiungere `pam_time` accanto a `pam_access` e restringere un utente all'orario d'ufficio in `/etc/security/time.conf`.

```
Exercise 1 – PAM Anatomy
  Exercise 1: All 13 checks passed
Exercise 2 – Password Policy (pam_pwquality)
  Exercise 2: All 5 checks passed
Exercise 3 – Account Lockout (pam_faillock)
  Exercise 3: All 3 checks passed
```

```
Exercise 4 – Resource Limits (pam_limits)
Exercise 4: All 5 checks passed
Exercise 5 – Time-based Access (pam_time)
Exercise 5: All 2 checks passed
Exercise 6 – Host/User Access Control (pam_access)
Exercise 6: All 3 checks passed
Exercise 7 – Custom Audit (pam_exec)
Exercise 7: All 4 checks passed
Exercise 9 – Centralized LDAP Authentication (sssd)
Exercise 9: All 8 checks passed
Exercises run:      8
Exercises passed:   8
Exercises failed:   0
```

`qlab test pam-lab` esegue ognuna di queste modifiche, verifica il comportamento che doveva produrre e ripristina i file originali — che è anche la risposta meglio documentata alla domanda "ma questo modulo cosa fa davvero".